

Python Topic 3 — Numbers & Operators

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert

Layer 1 — New to Python

Python has two main kinds of numbers:

```
age = 26          # int    → whole number
price = 19.99     # float  → number with a decimal point
```

You do math with the symbols you'd expect:

```
2 + 3    # 5    addition
10 - 4    # 6    subtraction
5 * 3     # 15   multiplication
10 / 2    # 5.0  division
```

i First surprise: `10 / 2` gives `5.0`, not `5`. In Python, `/` division **always** gives a decimal (float), even when it divides evenly.

Layer 2 — Intermediate (real code + how it works)

The full set of operators

```
10 + 3    # 13    add
10 - 3    # 7     subtract
10 * 3    # 30    multiply
10 / 3    # 3.333... true division → always float
10 // 3   # 3     floor division → drops the decimal
10 % 3    # 1     modulo → the remainder
10 ** 3   # 1000  exponent (power) → 10 to the power 3
```

Two operators worth memorizing (they behave differently from plain JS):

- `//` **floor division** — divides and rounds *down* to a whole number
- `**` **exponent** — power (JS has `**` too now; `Math.pow` is the old way)

JS comparison for the tricky ones

```
// JS
Math.floor(10 / 3) // 3 → you need Math.floor
10 % 3            // 1 → modulo same as Python
Math.pow(10, 3)   // 1000

# Python – built into the operators
10 // 3          # 3
10 % 3           # 1
10 ** 3          # 1000
```

Mixing int and float → result becomes float

```
5 + 2.0          # 7.0 (int + float = float)
type(5 + 2)       # <class 'int'>
type(5 + 2.0)     # <class 'float'>
```

Converting between types

```
int("42")        # 42    → string to int
float("3.14")     # 3.14  → string to float
str(42)           # "42"  → number to string
int(3.9)          # 3      → float to int (TRUNCATES, doesn't round!)
round(3.9)        # 4      → proper rounding
```

⚠ **Gotcha:** `int(3.9)` gives `3`, not `4` — it chops off the decimal, it doesn't round. Use `round()` when you want rounding.

Useful built-ins

```
abs(-5)           # 5      absolute value
round(3.14159, 2) # 3.14   round to 2 decimals
min(3, 7, 1)      # 1
max(3, 7, 1)      # 7
pow(2, 10)         # 1024  same as 2 ** 10
```

Layer 3 — Expert (internals & gotchas)

1. The float precision trap (the #1 gotcha)

```
0.1 + 0.2          # 0.30000000000000004  (!!)
0.1 + 0.2 == 0.3   # False  (!!)
```

This is the same in JavaScript. Floats are stored in binary (IEEE 754), and `0.1` can't be represented exactly in binary — just like `1/3` can't be written exactly in decimal.

 **For money, never use float.** Use the `Decimal` type — critical for your Razorpay/payment work. Always use `Decimal` (or integer paise/cents) for currency.

```
from decimal import Decimal
Decimal("0.1") + Decimal("0.2")  # Decimal('0.3')  ✓ exact
```

2. Integers have unlimited size (a big win over JS)

```
2 ** 1000  # a 302-digit number – computes perfectly, no overflow
```

 **JS contrast:** JavaScript numbers overflow past `Number.MAX_SAFE_INTEGER` (~9 quadrillion) and need `BigInt`. Python `int` is arbitrary-precision by default — it just grows as needed.

3. `//` floor division rounds toward negative infinity

```
7 // 2      # 3
-7 // 2     # -4  (!) not -3 – it rounds DOWN, toward negative infinity
```

`-7 / 2` is `-3.5`, and floor rounds down to `-4`. If you want truncation-toward-zero, use `int(-7 / 2)` → `-3`.

4. Booleans are secretly integers

```
True + True      # 2  (!) True is 1, False is 0
sum([True, False, True, True]) # 3 → handy for counting matches
isinstance(True, int) # True
```

✓ **Trick:** `sum(x > 5 for x in numbers)` counts how many values are greater than 5.

5. Chained comparisons (math-like ranges)

```
age = 25
18 <= age < 65  # True → reads like real math
# JS forces: age >= 18 && age < 65
```

Q: Difference between `/` and `//` ?

A: `/` is true division and always returns a float (`10/2 → 5.0`). `//` is floor division, returns the value rounded down toward negative infinity (`-7//2 → -4`).

Q: Why is `0.1 + 0.2 != 0.3` ?

A: Floats use IEEE 754 binary representation; `0.1` and `0.2` can't be stored exactly, so the sum is slightly off. For exact decimal math (money) use `Decimal` .

Q: Does Python have integer overflow?

A: No. `int` is arbitrary-precision and grows as needed — unlike JS, which caps at `MAX_SAFE_INTEGER` and needs `BigInt` .

Q: `int(3.9)` vs `round(3.9)` ?

A: `int(3.9)` truncates → `3` . `round(3.9)` rounds → `4` . `int()` never rounds; it drops the fractional part.

 **Memory hook:** `/` gives you decimals, `//` gives you whole numbers. Floats lie about money — use `Decimal` .

✓ **One-liner for interviews:** "Python has two numeric types you use daily — `int` (arbitrary precision, no overflow) and `float` (IEEE 754, subject to precision errors). `/` always returns float; `//` is floor division. For money, use `Decimal` , never float."